Modular .NET MVC Role & Permission Project

What this provides: a complete design + code snippets to build a modular ASP.NET Core MVC application (plug-and-play modules) that supports parent/grandparent menus, role-based URL permissions, multi-role users, menu generation by permission, permission checks (anonymous/authenticated/authorized), middleware, Razor UI visibility helpers, super-admin bypass, MySQL main DB, audit log (MySQL or Mongo), hourly log archiving, mobile-first modern UI (dark/light), login/logout/forgot-password/2FA email, notifications (permanent & temporary), blog module with comments/replies, cache-reset on role changes, dynamic DI/module/service discovery, and dynamic DB schema additions at startup (no EF migrations).

Table of contents

Open this document and follow each section. The doc contains copy-pasteable code snippets for `Program.cs`, `DbContext`, models, middleware, hosted services, repositories, services, and sample Razor tag helper.

1) Prerequisites

- .NET SDK 9 (or latest). Use `dotnet --version`.
- Visual Studio 2022/2023 or VS Code.
- MySQL server.
- MongoDB (optional) — if not provided, audit logs use MySQL.
- `Pomelo.EntityFrameworkCore.MySql` EF Core provider (works with EF Core).
- Tailwind CSS (for modern/mobile-first UI) or Bootstrap 5 (fallback).

2) Project structure (modular)

```
/ModularHost/                 <- main host (ASP.NET Core MVC)
  ModularHost.sln
  /src/
    /ModularHost.Web/        <- main MVC project (serves UI, static files)
    /Modules/                <- folder contains plug-and-play module projects
      /BlogModule/           <- example module (MVC area)
      /SomeOtherModule/
    /Shared/                 <- shared DTOs, utilities, helpers
  /db-scripts/
  README.md
```

How modules plug in: each module is a class library with MVC controllers/views/assets and a `ModuleStartup` static class for registration. The host dynamically loads assemblies from `Modules` folder at startup and registers controllers with `ApplicationPartManager`.

3) Database & ORM

- Use EF Core with Pomelo MySQL provider.
- Single `AppDbContext` that contains core tables. Modules may add their own tables by providing `IEntityTypeConfiguration<T>` classes detected at startup.
- Connection string in `appsettings.json` for MySQL and optional Mongo connection for audit logs.

appsettings.json (example)

```
{
  "ConnectionStrings": {
    "Default":
"Server=localhost;Database=modular_app;User=root;Password=YourPass;"
  },
  "Audit": {
    "UseMongo": true,
    "MongoConnection": "mongodb://localhost:27017",
    "MongoDatabase": "auditdb"
  }
}
```

4) Key domain models

- `User` (Id, UserName, Email, PasswordHash, IsSuperAdmin, ...) -- supports multiple roles
- `Role` (Id, Name, Description)
- `UserRole` (UserId, RoleId)
- `Menu` (Id, Title, Url, ParentId nullable, Order, IsVisible, Icon, ClaimType?)
- `RolePermission` (RoleId, Url, HttpMethod?)
- `Log` and `LogArchive` (common audit shape)

- `Notification` (Id, Title, Message, IsPermanent, TargetRoles, TargetUsers, CreatedAt, ReadBy list)
- `BlogPost`, `Comment`, `CommentReply`

Each model has `CreatedBy`, `CreatedAt`, `UpdatedBy`, `UpdatedAt` for audit fields.

5) Dynamic module loading & DI

- Host loads `.dll` files in `Modules` folder at startup.
- For each assembly load:
- Add as an `ApplicationPart` so MVC detects controllers and views.
- Scan for types that end with `Service`, `Repository`, `Controller` and register DI automatically (`Scoped` by default).
- Optionally call a `ModuleInitializer` static method if present.

This gives plug-and-play behavior: drop a compiled module into `/Modules` and restart host.

6) Repositories & Services (base classes)

- Provide `IRepository<T>` and `EfRepository<T>` implementing common `Add/Update/Delete/Get/Query`.
- Provide `IUnitOfWork` if you want explicit transaction scope.
- Provide `BaseService<T>` for shared operations.

Cache-aware permission service: `IPermissionService` returns allowed URLs/menus for a user (caches result keyed by user id). On role changes, the cache key for affected users is invalidated.

7) Authorization model & middleware

- Use cookie authentication and custom authorization middleware.
- Roles are not necessarily ASP.NET Roles: we use our `RolePermission` checks.
- Middleware checks request URL (and optionally HTTP method) and evaluates:
- If route is marked `AllowAnonymous` -> pass.
- Else if route requires authentication -> ensure user signed in.
- Else if route requires specific authorization -> check user roles and role permissions for URL.
- If user.IsSuperAdmin -> allow all.
- Also provide an attribute `[UrlAuthorize(Policy = AuthorizationPolicyType.Authorize)]` to declare intent (Anonymous / Authenticate / Authorize) on controllers/actions. Middleware reads attribute metadata.

Example flow: middleware runs after authentication and before MVC endpoint execution. It uses `IPermissionService.IsUrlAllowed(User, requestPath)`.

8) Razor helpers and view button visibility

- Provide a TagHelper / HtmlHelper extension: `@Html.ShowIfAuthorized("GET:/Admin/Users") { <button>...</button> }` or `[AuthorizeView]` component.
- Implementation queries `IPermissionService` (uses cache) and renders content accordingly.

9) Caching & cache reset on role/permission changes

- Use `IMemoryCache` for per-host caching.

- When role created/updated/deleted or role-permission changed, call `IPermissionCache.ResetRole(roleId)` which triggers removal of cached entries for users in that role. Provide `PermissionVersion` stamp stored in DB or distributed cache for multi-instance setups.

10) Audit logging (MySQL vs Mongo) + archive job

- `IAuditLogger` interface with implementation `MySqlAuditLogger` and `MongoAuditLogger`.
- Choose implementation at startup based on `appsettings:Audit:UseMongo`.
- Both write to `Log` table/collection and `LogArchive` table/collection.
- Hosted service `HourlyLogArchiverHostedService` runs every hour, moves items older than 1 hour (or configured threshold) from `Log` to `LogArchive`.
- If Mongo: use aggregation + copy + delete.
- If MySQL: `INSERT INTO LogArchive SELECT * FROM Log WHERE CreatedAt < @threshold; DELETE FROM Log WHERE CreatedAt < @threshold;` wrapped in transaction.

  Important: ensure archive queries are safe for large tables (batched) and have indexes on `CreatedAt`.

11) Notifications

- `Notification` entity stores permanent notifications. Temporary notifications are kept in memory (per-session) or broadcast via SignalR and not persisted.
- Provide admin UI to send notifications filtered by role(s), user(s), or all users.
- Use SignalR to push temporary notifications to connected clients. Permanent notifications saved and displayed in notification panel.

12) Blog module

- Example `BlogModule` with MVC controllers and Razor views for blog list (landing page), blog details, comment form, replies.
- `Comment` entity: `Id, PostId, ParentCommentId (nullable), UserId, Body, CreatedAt`.
- Moderation flags can be added.

13) UI: mobile-first, dark/light

- Use Tailwind CSS + small JS for dark/light toggle. Store preference in `localStorage` and a cookie.
- Provide a responsive sidebar that collapses for mobile; render menu based on `IPermissionService.GetMenuForUser(UserId)` which returns hierarchical menu tree (Parent -> Children -> Grandchildren).

14) Authentication: login, forgot password, 2FA via email

- Use cookie-based authentication with identity-like system (but custom to support MySQL schema). Store `PasswordHash` (use bcrypt via `BCrypt.Net-Next`), `TwoFactorEnabled`, `TwoFactorSecret` (if using app-based), or send one-time code via email.
- `EmailHelper` class to send email using SMTP credentials provided in `appsettings`.
- Forgot password: generate secure token, email link with token, token store in DB with expiry.

- 2FA (email): generate OTP, save short expiry in DB or distributed cache, email to user — user must enter OTP.

15) Deployment & running

- Build host, drop module DLLs into `Modules` folder, restart host.
- Use `dotnet publish` to create deployable artifacts.
- For multi-instance (scale-out), use Redis for distributed caching and SignalR backplane.

16) Security notes & caveats

- Dynamically altering DB schema at startup is risky in production — prefer migrations. If you choose to `ALTER TABLE` for added columns, ensure you have proper backups and a test environment.
- Caching permission results must consider distributed environments (use Redis or a version stamp persisted to DB).

17) Claude Premium instruction (how to use this with Claude)

If you want Claude Premium to review, document, or generate code for this project, copy the contents of this document and prompt Claude like:

```
You are given a detailed project specification and starter code. Provide:
1. A full file-by-file implementation plan for an ASP.NET Core 9 modular MVC project that meets these requirements.
2. Generate `Program.cs`, `AppDbContext`, core models, repository base class, permission middleware, one example module (Blog) with controllers and views, and a minimal front-end (Tailwind) layout.
3. For each generated file, include a short explanation and any required NuGet packages.
4. Point out security pitfalls and testing checks to run before deploying.

Here is the project spec: [paste the spec]
```

Claude will accept this and can produce code. Use safety: have it include small code chunks at a time.

Next steps I can do for you right now

- generate a full Visual Studio solution (`.sln`) + project files & scaffolded controllers + EF Core `AppDbContext` + models for the core tables and blog module.
- or generate just `Program.cs`, `AppDbContext`, models, middleware, and one sample controller + Razor view.

Tell me which one you want and I will scaffold it in this canvas and produce downloadable files.

If you want I can now scaffold the repository and produce a ZIP you can download and open in Visual Studio.